L03 – Bash Shell Scripting - Part 1

1. What is a shell?

So now you've been using Linux operating system for a couple of weeks. Things are a little different here than in the world of Mac or PC that you are likely accustomed to. One major difference is that you are playing around in a terminal, and typing directly into a command line. Getting started in a Linux environment is like going through mouse detox. Instead of clicking our way around, everything happens at the command line of the terminal. But, how are the commands we type interpreted? This depends on the ***shell***, where the shell is just a ***command-line interpreter***. That is, a shell is really just a computer program that reads what you type into the terminal and then interprets what to do with it.

There are a lot of different shells. The most common shells today seem to be the Bourne Again Shell (bash) and the C Shell, but there are some older ones you might encounter such as the Korn shell.

To see which shells are actually available on your system type:

```
>> cat /etc/shells
```

You can easily download and install others. In graduate school, all the computers we had used C-shell, or an improvement on C-shell called TCSH. This is what I learned, and it is something many geophysicists will use, but in my opinion Bash is better; it's more sophisticated and handles strings and numbers better. There are lots of guides to Bash scripting on the internet; here's one example: http://tldp.org/LDP/Bash-Beginners-Guide/html/index.html

2. What is a shell script?

Normally, when you are sitting at your terminal, the shell is interactive. This means the shell takes the command you type in and then it executes this command. This can be rather tedious if you want to do a larger number of commands in a specific order and maybe do it over and over again on different sets of data. Luckily, we can just write our sequence of commands into a text file, and then tell the shell to run all of the commands in this text file. This text file containing all of our commands is a **shell script**.

Let's make a simple one as an example. Open up a new file named **example.bash** with your favorite text editor and type the following:

```
#!/bin/bash

# the simplest shell script possible

clear
echo "I [heart] seismology"
```

After creating this file, type the following on the command line:

```
>> chmod +x example.bash
```

This will set the permissions for your new file **example.bash** such that you are allowed to execute it. You only need to do this once for a new file and not after every time you edit it.

Now you can execute the commands in this text file by typing:

```
>> ./example.bash
```

A couple notes on the above script.

Line 1: **#!/bin/bash** - this basically just says that I want to use the Bash shell to interpret these commands. Every Bash script must start out with this as the top-most line. You can add flags to this, for instance, when debugging, it is helpful to add the -x and -v flags to the first line, i.e.

#!/bin/bash -xv

The **-v** flag prints out each line in the script before it is executed, and the **-x** flag expands each line (such as adding in the content of variables). So when you run your script you'll see much more than you would otherwise.

Line 2: # **the simplest…** - you can add comments, and should frequently, to your scripts if you start the line out with the **#** symbol

Filename: example.bash – unlike on a windows machine Linux machines do not require you to have a file extension in most cases. However, it usually makes sense for people to adopt some kind of nomenclature so that you quickly know what kind of file you are dealing with. Hence, I usually use **.bash or .sh** to let me know that I have a Bash script.

Now, add the -xv flags to the first line and run **example.bash** again:

```
>> ./example.bash
```

See the difference? Note that the **-x** flag inside a bash script is different from the chmod **+x** command that we have to issue to make a shell script executable.

OK, now that we have that out of the way, type up the following script and see what it does. Call it **example2.bash**

```
#!/bin/bash
# Script to print user information who currently login ,
# current date & time

clear
echo "Hello $USER"
echo "Today is ";date

echo "Number of users logged in: " ; who | wc –l
```

```
echo "Calendar"
cal
```

Let's run this now.

```
>> chmod +x example2.bash
>> ./example.bash
```

3. Bash Variables

There are two types of variables:

(1) Environmental variables – that are created and maintained by the Linux system itself.

We saw one example of these in the example script above: **$USER**. Another example would be if you wanted to print out your home directory then you could type:

```
>> echo $HOME
```

You can also create your own variables in .bash_profile, e.g.

```
export DATA=$HOME/data
```

This is called an environmental variable. You can even access this new at the command line, as well as in a bash script, e.g.

```
>> cd $DATA
```

You can see all the environmental variables by typing

```
>> env
```

(2) User defined variables – that are created and maintained by the User.

Setting variables in a Bash script is done in two ways:

(a) String variables. String variables are just treated as a bunch of text characters. i.e., you cannot do math with them. String variables are created an equals sign as shown below. To access the variable, you use the **$** symbol. For instance, if you set **x=1**, and you want to set the variable y equal to x, you would type **y=$x**. See the examples below.

```
#!/bin/bash

x=1
y=2
class="seismology"
myfavoriteclass=$seismology
```

```
echo $x $y $class
echo $x + $y
echo "my favorite class is $myfavoriteclass"
echo myfavoriteclass
echo '$myfavoriteclasss'
```

Note that there can be no space on either side of the equals sign. Double quote are useful to make sure spaces are preserved. Use single quotes (the ' on the same key as the ") if you want all the characters between the single quotes to be taken literally. Here's a more thorough example of how this works: http://tldp.org/LDP/abs/html/varsubn.html

A really cool thing that can be done in shell scripts is running a command and assigning the output of the command to a variable. To do this, enclose the command in single back quotes .(On my keyboard this is the symbol under the tilde in the upper left hand part of the keyboard).

So, for instance, to save the date in a variable we can type **today=`date`**. Note that this can be done on the command line, for testing. For example:

```
>> today=`date`
>> echo $today
```

Try this script and see if it makes sense. Remember, if your script doesn't work, add -xv to the first line to help you debug it.

```
#!/bin/bash

date=`date`
echo "today is $date"
timezone=`date | awk ' {print $5} ' `
echo "our time zone is $timezone"
```

Summary of quotes:

Quotes	Name	Meaning
" "	Double Quotes	"Double Quotes" - Anything enclosed in double quotes removes the meaning of the characters (except \ and $). For example, if we **set arg = blah**, then **echo "$arg"** would result in **blah** being printed to the screen.
' '	Single quotes	'Single quotes' – Text enclosed inside single quotes remains unchanged (including $variables). For example, **echo '$arg'** would result in **$arg** being printed to the screen. That is, no variable substitution would take place.
` `	Back quote	`Back quote` - To execute a command. For example, **`pwd`** would execute the print working directory command.

(b) Numeric variables. Bash doesn't really do numbers well. To add integers, one puts the expression in double parentheses. Another way is to use the **let** command. Note that when a variable is part of a numerical expression, we don't add the **$** to it. To add floating point numbers (decimals) we have to pipe the expression we want evaluated to the program **bc**.

Adding integers:

```
b=$((1+2))
a=$((b+2))
let z=a+b
```

Adding decimals

```
c=`echo $a + 1.13 | bc`
```

Yeah, it's a little awkward, but it works well enough. And remember, when you are confused, just google "floating point addition bash" (for example). There are plenty of guides out on the internet.

Try this:

```
#!/bin/bash

a=2
b=$((z+3))
c=$((4+3))
let d=b+4
e = `echo $z + 3.14 | bc`
echo $a $b $c $e
```

What happens if you try:

```
set x = $x + 10
```

in the above script?

(c) Arrays of String Variables.

You can also use a single variable name to store an array of strings.

```
#!/bin/bash

days=(mon tues wed thurs fri)

echo ${days}
echo ${days[3]}
echo ${days[0]}
```

To echo an array variable, you need to use the curly brackets. These can be used for normal variables too. Note that bash starts counting at 0. See the output in the script above for **${days[0]}**, for instance.

As a special note: variables ***are*** case sensitive. For example, the three following combinations of the letters **n** and **o** are all considered to be a different variable by bash. This is important to remember as it is not alwaysthe case with other programming languages (e.g., in Fortran all three of these variable names would be considered to be the same variable).

```
no = 10
No = 11
nO = 12
echo $no $No $nO
```

As a final note on displaying shell variables it is often useful to concatenate shell variables:

```
#!/bin/bash

year=2010
month=12
day=30

output1=${year}_${month}_${day}
output2=${year}${month}${day}

echo $output1
echo $output2
```

Note that we use the **{ }** brackets in this example. This is because if I just type **$year_** then the shell would look for a variable called **year_**.

4. Command Line Arguments

It is often useful to be able to grab input from the command line or to read user input. The next example shows a simple way to interactively get information and set the result to a variable.

```
#!/bin/bash

echo -n "How many records in this file do you want to skip? "
read nlines

echo $nlines
```

Bash refers to arguments typed on the command line as $1, $2, etc.

```
#!/bin/bash

echo "Now lets perform some kind of action on file: $1"
echo "note" '$1' "is $1"
```

If I named this script: **action.bash**

and we want to perform the action on the file **foo.txt**

then we need to type:

```
>> action.bash foo.txt
```

on the command line to make this work. This is really useful when we want to make generalized scripts that don't require editing the variable names every time we want them to run.

5. Redirection of standard output/input

The input and output of commands can be sent to or received from files using redirection. Some examples are shown below:

date > datefile

The output of the **date** command is saved into the contents of the file, **datefile**.

a.out < inputfile

The program, **a.out** receives its input from the input file, **inputfile**.

sort gradefile >> datafile

The **sort** command returns its output and appends it to the file, **datafile**.

5.1 stdin, stdout, and stderr

When bash starts, it opens three file descriptors, stdin, stdout, and stderr. These can point to a file or another input/output device , such as a terminal. Typically, when bash starts, all these three file descriptors point to the terminal.

stdin is the data going into a program.
stdout is the output of a program, typically the results. In bash redirections of stdout are associated with the number 1.
stderr is the error or other diagnostic messages output by a program. In bash redirections of sdterr are associated with number 2.

5.1.1. stdin

If you want to send a file called **data.txt** to the input of a program, you could type

my_file < data.txt

There is something called a "here" document, which creates a temporary file that is used to input into a program. For instance,

cat << EOF

sends input from a file that you make to cat, until the EOF is reached.

For instance, type

```
> cat << EOF
> this
> is
>a
> here
>file
>EOF
```

A use for here files in seismology is through a program we use (creatively) called "Seismic Analsysis Code" (SAC). A herefile for sac would look something like this and would send the commands listed until EOF is reached.

```
#!/bin/bash

sac << EOF
r infile.sac
qdp off
ppk
q
EOF
```

5.1.2 stdout and stderr

Say you want to redirect the output of a command to a file, you could do this.

who > file or **who 1>file**. Both of these are ways of sending the *stdout* of **who** to a file.

If you have a program that displays error messages and you want to send these to a file, you'd type

```
my_program 2> program_errs.txt
```

If you want to send stdout to stdin, you'd type

```
my_program 2>&1
```

And if you wanted to send both of these to a file, you'd type

```
my_program > all_program_outputs.txt 2>&1
```

This says send stdin to all_program_outputs.txt, and then send stdout to stdin, which is set to all_program_outputs.txt

This is kind of a complicated command, but fortunately there's a shorthand version

```
my_program &> all_program_outputs.txt
```

will do the same thing.

These commands can be quite useful when you want to capture all of the output of a program. If you don't want to see any output, a handy way to do it is:

my_program &> /dev/null

/dev/null is sort of a black hole that eats anything you give it.

7. Homework

1) Write Bash script that will add the current date to the end of the file name. So, if your bash script is called Schutt_hw3.bash, the filename is called my_file.txt, and the date is Dec. 25, 2010, then you would type

>> Schutt_hw3.bash my_file.txt

and the file name would be changed to my_file.txt.20101225.

You may want to have a look at the man page for date.

Call the bash script yourlastname_yourfirstname_hw3.bash and put it in our class assignment dropbox.

8.0 Acknowledgements

This lab is based on an excellent series of computational geophysics write-ups by Michael Thorne of the University of Utah. He's got really cool animations on his website too. http://web.utah.edu/thorne/